Prayush B Menon

Phone Number: 628-263-2031 | Email: bmenonprayush@gmail.com
LinkedIn: linkedin.com/in/prayush-menon | GitHub: github.com/k0-R0 | Portfolio: k0-r0.github.io

CAREER OBJECTIVE

Embedded Systems & Firmware Engineer with an ECE background and hands-on expertise in bare-metal C/C++ development on PIC microcontrollers. Strong foundation in hardware protocols (CAN, UART, SPI, I2C), memory-efficient data structures, Linux systems programming, and debugging with GDB. Experienced in end-to-end hardware-software integration, multi-threaded POSIX environments, and low-level firmware engineering.

TECHNICAL SKILLS

Languages: C / C++, Java
Microcontrollers & Protocols: PIC18F4580, UART, I2C, SPI, CAN Protocol
Developer Tools & OS: Linux (Fedora), GDB, Make, Git, Neovim, Vim, Cursor
Software Testing & Management: Selenium, Playwright, Test Automation, JIRA, Octane

PROJECTS AT EMERTXE

Minishell (Unix Command Shell) Github

C, Linux Systems Programming, IPC, Process Management, Signals

- **Project brief:** Built a custom Unix command shell in C to understand how Linux manages processes and system calls. The shell supports executing programs, internal built-in commands, multi-stage pipes, signal handling, custom prompts, and job control.
- **Key challenges & Learnings:**
  - Chained multiple commands using `pipe()` and redirected standard streams with `dup2()`, synchronizing process completion using `waitpid()`.
  - Added signal handling for `SIGINT`, `SIGTSTP`, and `SIGCHLD` to suspend foreground jobs and resume them in the background or foreground (`bg`/`fg`).
  - Used a singly linked list to track process IDs and execution states for background and stopped jobs.
  - Engineered built-in commands (`cd`, `pwd`, `exit`, `echo`) with environment variable parsing (`$PATH`, `$?`) and asynchronous zombie process reaping.

Multithreaded TFTP Server & Client Github

C, UDP Sockets, POSIX Threads, Semaphores, Systems Programming

- **Project brief:** Built a multithreaded TFTP file transfer server and client in C over UDP sockets. The server handles multiple file uploads and downloads at the same time by spawning worker threads, using a counting semaphore to prevent server overload and a stop-and-wait protocol for reliable delivery over UDP.
- **Key challenges & Learnings:**
  - Capped active worker threads at 10 using a counting semaphore (`sem_t`); used non-blocking `sem_trywait()` so the main server loop immediately replies with a "Server is busy" packet when full without freezing.
  - Gave each worker thread its own ephemeral UDP socket to avoid packet collisions, and configured threads as detached (`PTHREAD_CREATE_DETACHED`) so memory and resources are freed automatically on exit.
  - Built a stop-and-wait acknowledgment protocol with custom packet structs (`cmd_packet`, `data_packet`, `ack_packet`) to track 512-byte block sequence numbers and resend unacknowledged blocks over UDP.
  - Supported 3 transfer modes—standard binary (`MODE_OCTET`), single-byte blocks, and a mail mode that converts `\n` to `\n\r`—along with an interactive CLI client for batch file transfers.
  - Implemented timeout retransmission logic and packet error handling (`ERROR` opcode) to recover gracefully from dropped UDP datagrams without deadlocks.

Concurrent HTTP/1.1 Server Github

C, POSIX Sockets, pthreads, Semaphores, HTTP/1.1, Systems Programming

- **Project brief:** Built a lightweight, concurrent HTTP/1.1 web server in C using POSIX sockets. Implemented a producer-consumer thread pool with a bounded circular connection queue and synchronization primitives to handle simultaneous client connections, dynamic routing, header inspection, and file transfers.
- **Key challenges & Learnings:**
  - Designed an 8-worker thread pool sleeping on a counting semaphore (`sem_t`), waking instantly on connection dispatch to achieve zero idle CPU overhead.
  - Implemented a thread-safe circular ring buffer (capacity 64) synchronized via `pthread_mutex_t` to queue client file descriptors from the `accept()` loop.
  - Engineered an HTTP/1.1 protocol parser to extract request methods, paths, and headers, supporting dynamic route dispatching (e.g., `/echo/{str}`, `/user-agent`).
  - Added static file delivery with automatic extension-based MIME type resolution (`.html`, `.png`, `.jpg`, `.txt`) and disk-backed `POST` file uploads with zero socket leaks.

Car Black Box (Event Data Recorder) Github

Embedded C, PIC18F4580, I2C, UART, ADC, EEPROM, RTC, CLCD

- **Project brief:** Built an event data recorder on a PIC18F4580 microcontroller in Embedded C to log vehicle telemetry during driving and crash events, saving the 10 latest events in an external EEPROM with a CLCD menu and data export to PC over UART.
- **Key challenges & Learnings:**
  - Wrote bare-metal I2C functions to communicate with the DS1307 real-time clock and AT24C04 EEPROM on a shared two-wire bus.
  - Implemented a circular buffer to store and retrieve the 10 latest events in EEPROM across resets.
  - Integrated drivers for an ADC potentiometer for speed sampling, matrix keypad, 16x2 CLCD menu, and UART export to PC.
  - Engineered secure password authentication with attempt counters, lockout timers, and non-volatile credential updates directly in EEPROM.

Image Steganography (LSB) Github

C, Bit Manipulation, Image Processing, Binary File I/O

- **Project brief:** Built a command-line tool in C to hide and extract text messages inside 24-bit BMP images using the Least Significant Bit (LSB) technique. The program verifies the BMP header and embeds message bits into RGB pixel bytes without visibly changing the image.
- **Key challenges & Learnings:**
  - Used bit masking and shifting to replace the least significant bits of image pixel data with message payload bits.
  - Protected the 54-byte BMP header to keep file metadata intact and ensure the modified image opens in standard image viewers.
  - Embedded a 2-byte magic string and 32-bit payload size header into pixel data to authenticate files during extraction and prevent buffer overruns.
  - Implemented error handling and boundary verification to prevent buffer overflows during payload encoding and decoding.

Car Dashboard (Automotive CAN Simulator) Github

C, Embedded Systems, PIC18F4580, CAN Protocol, LCD

- **Project brief:** Built an automotive dashboard simulator using two PIC18F4580 microcontrollers communicating over a CAN bus. One microcontroller acts as an engine ECU broadcasting speed, RPM, and gear status, while the second acts as a dashboard ECU and renders the data on an LCD.
- **Key challenges & Learnings:**
  - Configured PIC18F4580 CAN registers, message identifiers, and interrupts to broadcast vehicle telemetry frames.
  - Wrote display logic on the receiver ECU to filter incoming CAN frames, unpack data bytes, and update the dashboard LCD in real time.
  - Simulated driving state changes and sensor inputs to test synchronized communication across the CAN bus.
  - Configured CAN acceptance filter masks to isolate engine telemetry packets and prioritized critical alert frames with hardware arbitration.

Lexical Analyzer for C Github

C, Lexical Analysis, Tokenization, Binary Search, File I/O

- **Project brief:** Built a C source code tokenizer that reads files character-by-character, strips comments and preprocessor lines, and groups code into keywords, identifiers, constants, strings, and operators.
- **Key challenges & Learnings:**
  - Used binary search ($O(\log N)$) over sorted symbol arrays to quickly look up and identify C keywords and operators.
  - Built parser state logic to cleanly remove single-line (`//`) and multi-line (`/* */`) comments while preserving line numbers.
  - Added syntax validation to catch unmatched parentheses, brackets, and braces (`()`, `{}`, `[]`), as well as unclosed strings.
  - Recorded precise line and column numbers during tokenization to generate detailed compiler-style diagnostic syntax error reports with exact indicators.

Inverted Search Engine Github

C, Hash Tables, File I/O, Data Structures

- **Project brief:** Built a text indexing and search engine in C. The program scans text files, builds an inverted index using a custom hash table, and maps each word to a linked list of file occurrences for fast keyword lookups.
- **Key challenges & Learnings:**
  - Implemented a hash table with linked-list collision handling to index and retrieve search terms in $O(1)$ average time.
  - Parsed multiple text files to track word counts, file names, and sentence positions without duplicating memory.
  - Added file persistence routines to save the generated database to disk and reload it without re-parsing files.
  - Supported multi-file query searches with Boolean operators (`AND`, `OR`) and dynamic index updating as new text files are added to the corpus.

Source2HTML (C Source to HTML Syntax Highlighter) Github

C, State Machines, Lexical Analysis, File I/O, HTML/CSS

- **Project brief:** Built a C program that converts C source code into syntax-highlighted HTML files. It scans code using a finite state machine, classifies tokens (keywords, comments, strings, preprocessor lines), and wraps them in CSS-styled HTML tags.
- **Key challenges & Learnings:**
  - Built a state machine with distinct states (idle, string, comment, directive, number) to accurately parse complex C syntax.
  - Mapped tokens to CSS color classes to highlight data types, control keywords, and preprocessor directives distinctly.
  - Escaped special HTML characters (`<`, `>`, `&`) to prevent browser rendering errors, and added an option for line numbering.
  - Added support for customizable CSS syntax themes, line numbering gutters, and entity escaping for nested multi-line comments and literal quotes.

MP3 Tag Reader & Editor Github

C, Binary Parsing, File I/O, ID3v2 Tags

- **Project brief:** Built a command-line tool in C to inspect and edit ID3v2 metadata (title, artist, album) inside MP3 files without altering the audio data.
- **Key challenges & Learnings:**
  - Parsed ID3v2.3 binary frame headers and calculated variable-length frame sizes to navigate file offsets.
  - Handled big-endian byte conversions to accurately read frame size descriptors from binary files.
  - Safely updated tag metadata in-place within the file header without corrupting the underlying MP3 audio stream.
  - Implemented dynamic frame resizing algorithms to append expanded metadata tags without truncating audio frames or misaligning sync headers.

Arbitrary Precision Calculator (APC) Github

C, Data Structures, Algorithms, Dynamic Memory Allocation

- **Project brief:** Built an arbitrary-precision calculator in C using doubly linked lists, allowing mathematical operations on numbers of any size without being limited by 64-bit data types.
- **Key challenges & Learnings:**
  - Wrote arithmetic algorithms for addition, subtraction, multiplication, and division across linked list nodes, handling carries and borrows digit-by-digit.
  - Handled dynamic memory allocation and cleanup carefully to ensure zero memory leaks during large arithmetic calculations.
  - Handled edge cases including sign management (positive/negative results), zero padding, and division-by-zero checks.
  - Optimized node digit compression and sign management, implementing high-speed division by repeated subtraction with division-by-zero checks.

Red-Black Tree (Self-Balancing BST) Github

C, Data Structures, Algorithms, Tree Rotations, Memory Management

- **Project brief:** Built a self-balancing Red-Black Binary Search Tree in C based on CLRS algorithms. Implemented core operations including insertion, deletion, and searching with logarithmic height balancing, maintaining Red-Black invariants through color fixups and tree rotations.
- **Key challenges & Learnings:**
  - Designed an explicit sentinel (`NIL`) node architecture to eliminate NULL-pointer edge cases during rotations and parent/child boundary transitions.
  - Implemented left and right tree rotations with node recoloring routines for `rb_insert`, resolving red-red violations across uncle cases.
  - Engineered `rb_delete` with subtree transplantation (`rb_transplant`) and multi-case deletion fixups to handle black-height deficits (double-black cases).
  - Built recursive in-order, pre-order, and post-order traversals, $O(\log N)$ min/max and successor/predecessor queries, and an interactive CLI test harness with leak-free memory cleanup.

Address Book Github

C, Data Structures, File I/O, CLI

- **Project brief:** Built a command-line contact management tool in C to store, search, edit, and delete contacts, saving all entries to a CSV file for persistent storage.
- **Key challenges & Learnings:**
  - Organized contacts in dynamically allocated arrays with search and sort functions by name, phone, and email.
  - Handled user input safely with string bounds checking to prevent buffer overflows and invalid entries.
  - Built file I/O routines to save contact data in a structured CSV format and reload it when the application starts.
  - Engineered multi-field search algorithms supporting case-insensitive lookups, data pagination, and CSV backup validation against file corruption.

EDUCATION & TRAINING

Emertxe Information Technologies Bangalore, India

Advanced Embedded Systems Training

- Undergoing a training program on embedded systems.

Amrita School of Engineering Kollam, Kerala

B.Tech in Electronics and Communications Engineering (CGPA: 8.1/10)

COURSE WORK

Data Structures & Algorithms (DSA), Microcontrollers, Embedded Systems, Operating Systems, Computer Networks